

ByteBridge: A Diagnostic Study of Input-Side Byte Interfaces for Frozen Language Models

Anonymous authors

Paper under double-blind review

Abstract

Byte-level input is attractive because bytes can represent spelling noise, rare Unicode, multilingual scripts, code, and other text that is awkward for fixed subword tokenizers. A practical shortcut would be to keep an existing language model frozen and train only a small adapter that turns input bytes into something the model can use. We test this shortcut with ByteBridge, a diagnostic study on a frozen Qwen2.5-0.5B causal LM. The input side is byte-based; the target side still uses the model’s tokenizer so that every system is scored on the same target tokens. We compare the native tokenizer baseline with a ladder of byte interfaces, starting with byte embeddings at the input layer and ending with byte-derived prefixes injected into every transformer layer. All runs reuse the same splits, freeze the LM and LM head, and verify zero backbone checksum drift. The result is negative but useful. Byte adapters consistently help on typo-noise prompts, but they do not match native tokenization on clean text or on Unicode, multilingual, code, and tokenizer-stress inputs. The best input-layer adapter has $1.84\times$ the clean loss of the tokenizer baseline. Soft-prefix and all-layer KV-prefix injection improve this to $1.76\times$ and $1.61\times$, meaning the best run still has 61% higher clean loss than native tokenization. These runs miss our $1.5\times$ promising threshold and never beat the tokenizer baseline outside typo noise. These results suggest that a frozen LM does not merely need text-like vectors. It also expects the units, boundaries, and positions produced by its tokenizer, and small byte adapters do not recover this structure well enough.

1 Introduction

Subword tokenizers are the front door of most language models. They turn raw text into the discrete units that the model was trained to read. This works well for common text, but it can be brittle. A small typo may change the segmentation. Rare Unicode characters may become many tokens. Code, file paths, long identifiers, and multilingual text may be split in ways that look very different from clean English.

Byte-level modeling avoids out-of-vocabulary inputs because every string can be written as bytes. Models trained for byte or character input have shown strong robustness and multilingual coverage (Xue et al., 2022; Clark et al., 2022; Tay et al., 2022; Yu et al., 2023). But most useful LMs today are not byte models. They were trained with a fixed tokenizer. A tempting question is whether we can keep such a model frozen and attach a small byte adapter in front of it.

This paper asks a narrow version of that question: can a small byte adapter give useful input context to a frozen tokenizer-based LM, without badly hurting clean-text performance? We study this with ByteBridge on Qwen2.5-0.5B (Qwen Team, 2024). The LM backbone, input embedding table, and LM head are always frozen. Only byte-side adapters are trained. The output side still uses the Qwen tokenizer, so this is not a fully tokenizer-free LM. It is a test of whether the input side of an existing LM can be replaced or augmented by bytes.

There are two plausible outcomes. The optimistic view is that a byte adapter can map UTF-8 bytes into continuous vectors that a frozen LM can use. This seems especially plausible if

those vectors are aligned to token embeddings or inserted through prompt- or prefix-tuning interfaces (Li and Liang, 2021; Lester et al., 2021). The pessimistic view is that a tokenizer-based LM learns more than a table of token embeddings. It also learns the units, boundaries, order, and attention patterns created by its tokenizer. A small adapter may not reproduce all of that.

We evaluate this tension with a diagnostic ladder. Each step tests one possible reason for failure. We start with fixed-patch byte latents passed through inputs_embeds. We then test embedding distillation, token reconstruction with fixed, heuristic, and oracle token boundaries, byte-derived soft prefixes, and per-layer KV-prefix injection using Qwen’s cache API. The same evaluation suite is reused throughout: clean English, typo noise, Unicode stress, multilingual text, code, and tokenizer-stress strings.

The main result is negative. Byte adapters consistently beat the tokenizer baseline on typo-noise prompts. But every byte adapter is still much worse on clean text, and the native tokenizer remains better on Unicode, multilingual, code, and tokenizer-stress inputs. Deeper injection helps: soft prefixes improve several stress buckets, and all-layer KV-prefix gives the best clean retention, $1.61\times$ the tokenizer baseline. This is progress, but it still misses our $1.5\times$ promising threshold and is far from the $1.2\times$ strong-pass threshold.

Our contributions are:

- We introduce a controlled benchmark and evaluation protocol for input-side byte adapters on frozen tokenizer-based LMs, with paired conditional targets and fixed splits.
- We provide a phase-by-phase diagnostic study of byte adapters, including input replacement, embedding distillation, token reconstruction, boundary-aware patching, soft prefixes, and KV-prefix injection.
- We report an artifact-backed negative result: in this frozen-Qwen2.5-0.5B, single-GPU, small-adapter setting, byte adapters improve typo robustness but do not become competitive replacements for native tokenized input.

2 Problem Setting

Let M be a pretrained causal LM with tokenizer T , input embedding matrix E , transformer F , and LM head H . In all experiments, E , F , and H are frozen. Given an input string x and target string y , the native tokenizer baseline computes

$$\mathcal{L}_{\text{tok}}(x, y) = -\sum_t \log p_M(y_t \mid T(x), y_{<t}), \quad (1)$$

where y_t are target tokens under the same tokenizer. The byte variants replace or augment the conditioning on x with continuous states $A(b(x))$. Here $b(x)$ is the UTF-8 byte sequence and A is the trainable adapter. The target side remains tokenized. This keeps the comparison fair: every system is scored on the same target tokens.

This paired conditional setup is important. Comparisons can be misleading if one system is scored on bytes and another on tokens, or if noisy inputs change the supervised target. We avoid that problem by keeping y fixed. For noisy examples, x is the perturbed string and y is the clean target suffix. Loss is computed only over target tokens, not over byte-prefix or adapter positions.

The main metric is per-bucket target-token loss. We also save perplexity in the experiment artifacts, but focus on loss because stress-bucket perplexities can be very large. To measure clean-text degradation, we use

$$\text{clean retention}(A) = \frac{\mathcal{L}_A(\text{clean})}{\mathcal{L}_{\text{tok}}(\text{clean})}. \quad (2)$$

Before running the main diagnostic ladder, we set two practical thresholds: adapters with at most 50% higher clean loss than the tokenizer baseline are treated as promising ($\text{clean retention} \leq 1.5$), and adapters with at most 20% higher clean loss are treated as

Split	examples	avg. input bytes	avg. input toks	avg. target toks
train	5168	121.9	34.1	26.1
validation	1188	99.5	30.9	25.7
test	2028	100.3	30.9	25.8

Table 1: Phase 2 split sizes reused for all later phases. The test set contains 500 clean-English examples; 200 examples each for light, medium, and heavy typo noise; 128 Unicode-stress examples; 120 code examples; 120 tokenizer-stress examples; and 80 examples for each multilingual bucket.

90 strong (clean retention ≤ 1.2). We also record per-bucket wins against the tokenizer base-
 91 line, trainable parameter counts, throughput, peak GPU memory, and checksum drift of
 92 frozen model tensors.

93 3 Evaluation Suite

94 We construct a small, reproducible suite that stresses tokenizer-unfriendly inputs while
 95 remaining feasible in a single-GPU setting. The train/validation/test manifests are fixed
 96 after construction and reused across all phases. Each record contains an identifier, bucket,
 97 input text, target text, source, perturbation metadata, seed, and clean-example identifier
 98 where applicable. Test examples are never used for training.

99 The test buckets are:

- 100 • Clean English: short natural-language, Wikipedia-like, instruction-like, and QA-like
 101 snippets.
- 102 • Typo noise: clean English with random deletion, insertion, substitution, adjacent
 103 swaps, repeated characters, whitespace perturbations, and casing noise at light,
 104 medium, and heavy levels.
- 105 • Unicode stress: emoji, accented characters, combining marks, zero-width join-
 106 ers, full-width characters, mathematical symbols, punctuation variants, and
 107 normalized/non-normalized mixtures.
- 108 • Multilingual text: short Chinese, Japanese, Korean, Arabic, Hindi, Russian, and
 109 Latin-script accented examples.
- 110 • Code: Python, JavaScript, JSON, shell commands, stack traces, regexes, SQL, and
 111 configuration snippets.
- 112 • Tokenizer-stress strings: URLs, file paths, package names, UUIDs, base64-like and
 113 hexadecimal strings, logs, long identifiers, and mixed separators.

114 Table 1 gives the split sizes. The benchmark is synthetic and curated, not a claim about
 115 natural text distributions. Its role is to make failure cases visible under controlled conditions.

116 The native tokenizer baseline is the frozen Qwen2.5-0.5B evaluated directly on this paired
 117 conditional test set. Its clean-English loss is 0.3089. This baseline is intentionally strong
 118 because it uses the tokenizer and embedding conventions the model saw during pretraining.

119 For reproducibility, every training and evaluation script writes its config, environment meta-
 120 data, training metrics, per-bucket summaries, and adapter checkpoint. All reported LM runs
 121 keep the frozen LM trainable parameter count at zero and the frozen-model checksum delta
 122 at zero.

123 4 Diagnostic Adapter Ladder

124 4.1 Input-layer byte latents

125 The simplest ByteBridge adapter groups bytes into fixed-size patches and maps them into L
 126 latent vectors in the LM hidden dimension. These latents are passed through `inputs_embeds`

as the prompt representation. This tests the most direct idea: replace tokenized prompt embeddings at the input layer. We train clean-only and noise-augmented variants; the strongest Phase 2 run uses $L = 32$ and noise-augmented training for 2000 steps.

4.2 Embedding distillation

One possible failure is that byte latents are simply not shaped like Qwen token embeddings. To test this, we align byte latents to frozen token embeddings before LM training. We use fixed token-span alignment, where token embeddings are truncated or pooled to L spans, and pooled sequence alignment, where byte and token representations are mean/max pooled. The adapter is then fine-tuned with the same frozen-LM conditional objective.

4.3 Token reconstruction and boundary-aware patching

Embedding MSE may be too weak. The LM may need to know which token-like unit a span represents, not just receive a nearby vector. We therefore build a token-byte mapping tool that maps tokenizer tokens to UTF-8 byte spans. For Qwen on this suite, tokenizer offset mapping yields zero unmapped tokens. We test three boundary modes: fixed byte patches, heuristic tokenizer-free boundaries based on UTF-8/script/punctuation/whitespace changes, and oracle tokenizer-derived byte spans. The oracle mode is a diagnostic upper bound, not a tokenizer-free method.

During reconstruction pretraining, a classifier predicts tokenizer token IDs from byte/span latents. The classifier head is discarded for LM evaluation; only the adapter initializes the frozen-LM conditional training run.

4.4 Soft prefix injection

Input replacement may be too strict because it forces byte latents to behave like token embeddings at prompt positions. Soft prefix injection uses bytes differently. It compresses the input bytes into P learned prefix embeddings, concatenates them before tokenized target embeddings, and computes loss only on target positions:

$$\text{inputs_embeds} = [A_{\text{prefix}}(b(x)); E(T(y_{<t}))]. \quad (3)$$

This keeps native target-side teacher forcing while giving the frozen LM byte-derived context.

4.5 KV-prefix injection

Finally, we test whether bytes must be injected deeper than the input layer. We probe Qwen’s DynamicCache API and construct byte-derived key/value tensors with shape $[B, H_{kv}, P, d_h]$ for selected layers. Qwen2.5-0.5B uses 24 layers, hidden size 896, 14 attention heads, 2 key/value heads, and head dimension 64. We test first-4-layer and all-layer injection. For first- N runs, non-injected layers receive zero K/V tensors of the same prefix length to keep cache sequence lengths consistent across layers; this is a diagnostic compromise, not an optimal prefix-tuning design.

5 Results

5.1 Main comparison

Table 2 summarizes the best representative run from each phase. Lower loss is better. A clean retention of $1.61\times$ means 61% higher clean loss than the tokenizer baseline. The pattern is consistent: each later diagnostic improves at least one aspect of the previous interface, but none approaches native tokenizer conditioning on clean text. The best input-layer reconstruction run reaches $1.84\times$ clean retention. Soft prefixes improve this to $1.76\times$, and all-layer KV-prefix reaches $1.61\times$. All remain above the $1.5\times$ promising threshold.

Stage	Representative run	clean loss	clean retention	wins vs. tokenizer
Tokenizer	native Qwen tokenizer	0.3089	1.00	–
Phase 2	fixed input, noise, $L = 32$	0.6770	2.19	typo L/M/H
Phase 3	distill then LM, $L = 32$	0.6863	2.22	typo L/M/H
Phase 4	fixed recon then LM	0.5669	1.84	typo L/M/H
Phase 5	soft prefix, $P = 64$	0.5441	1.76	typo L/M/H
Phase 6	KV prefix, $P = 32$, all layers	0.4985	1.61	typo L/M/H

Table 2: Best representative runs across the diagnostic ladder. “Typo L/M/H” denotes the light, medium, and heavy typo-noise buckets. No byte-interface run beats the tokenizer baseline outside typo noise.

Bucket	tokenizer baseline	Phase 2 best	Phase 4 fixed recon	soft prefix p64	KV prefix p32 all
clean English	0.3089	0.6770	0.5669	0.5441	0.4985
code	0.3876	2.4261	2.0058	0.7878	1.3462
tokenizer stress	0.4806	3.1706	2.8533	1.5118	1.5969
Unicode stress	0.5614	3.8226	2.3900	0.9630	0.9809
typo light	1.3838	0.7454	0.6416	0.5817	0.5167
typo medium	1.8428	0.7786	0.6626	0.6097	0.5298
typo heavy	2.8353	0.8203	0.7749	0.6686	0.5986
multilingual zh	0.6085	4.5419	4.7134	4.9432	11.1217
multilingual ar	0.3806	3.9973	4.4678	5.4298	10.7503

Table 3: Selected test losses. Lower is better. Byte-derived adapters consistently help typo-noise buckets but remain worse than native tokenization on clean and non-typo stress categories.

5.2 Bucket-level behavior

Table 3 shows selected bucket losses. The tokenizer baseline remains strongest on clean English, code, tokenizer-stress, Unicode, and multilingual text. The byte adapters are strongest on typo noise. This is the one robust positive signal across phases: when the prompt is corrupted but the target is clean, byte adapters can learn a denoising-style context that degrades less than native tokenized conditioning.

Deeper injection helps non-typo stress relative to earlier byte variants. Soft prefix p64 reduces code loss from 2.0058 under fixed reconstruction to 0.7878, tokenizer-stress from 2.8533 to 1.5118, and Unicode from 2.3900 to 0.9630. However, these losses remain worse than tokenizer baseline losses of 0.3876, 0.4806, and 0.5614. All-layer KV-prefix gives the best clean loss but is worse than soft prefix p64 on code and tokenizer-stress, and multilingual losses remain very poor.

5.3 What each diagnostic tells us

Fixed byte patches are insufficient. The Phase 2 input-layer adapter confirms engineering feasibility: gradients flow through inputs_embeds, adapter loss decreases, LM trainable parameters remain zero, and checksum drift is zero. But the best Phase 2 clean retention is $2.19\times$, with severe degradation on Unicode, code, multilingual, and tokenizer-stress buckets. This provides evidence against the simplest fixed-patch interface as a competitive tokenizer replacement in our setting.

Embedding alignment is not enough. Projection alignment works as an auxiliary objective: distillation loss drops from 0.9583 to 0.0637 for $L = 32$ and from 0.9654 to 0.0752 for $L = 64$. Downstream LM evaluation does not improve: the $L = 32$ distill-then-LM run has $2.22\times$ clean retention. This suggests that matching token embeddings geometrically is not enough; the frozen LM also appears to rely on token identity and sequence structure.

Token identity is learnable, but not sufficient. Token reconstruction gives a stronger diagnostic. Fixed reconstruction reaches validation clean top-1 accuracy 0.8669; oracle-boundary reconstruction reaches 0.9945. This confirms that token identity can be recovered from byte spans, especially when token boundaries are known. Yet downstream LM clean retention remains poor: fixed reconstruction improves to $1.84\times$, while oracle-boundary reconstruction is $2.20\times$. Oracle boundaries improve Unicode/code/stress losses relative to Phase 2, but they do not repair clean-text conditioning. Therefore boundary and identity errors are real bottlenecks, but not the only bottlenecks.

Deeper injection helps, but not enough. Soft prefix and KV-prefix variants test whether the frozen LM needs byte information as attention context rather than replacement input embeddings. The answer is partially yes. Soft prefix p64 dramatically improves code, Unicode, and tokenizer-stress losses over input-layer variants. All-layer KV-prefix gives the best clean retention, $1.61\times$, and preserves typo-noise gains. But neither reaches $1.5\times$ clean retention, and neither beats the tokenizer baseline on non-typo stress buckets. The tradeoff is also not monotonic: KV-prefix improves clean loss over soft prefix, but soft prefix remains better on code and tokenizer-stress.

6 Discussion

The negative result is informative because each variant addresses a plausible explanation for the previous failure. The ablations narrow the failure mode. Embedding alignment, oracle token boundaries, and deeper prefix injection each help only partially. If the issue were simply that byte latents looked unlike token embeddings, embedding distillation should have improved downstream loss. It did not. If the issue were only boundary segmentation, oracle token boundaries should have substantially closed the clean retention gap. They did not. If the issue were only shallow input injection, all-layer KV-prefix should have approached native token conditioning. It improved clean retention but remained far from the tokenizer baseline.

Our interpretation is that frozen tokenizer-based LMs depend on several tokenizer-created conventions at once. Native token embeddings carry local text content, but the model also expects familiar token units, familiar boundary lengths, familiar position steps, and familiar attention patterns. A small byte adapter trained after the fact can learn useful robust cues, especially for typo-noise denoising, but it does not recreate the full input interface.

The multilingual failures are especially important. They do not show that byte modeling is bad for multilingual text; byte-trained models are often attractive precisely because they avoid vocabulary coverage problems. Instead, they show that our small adapters do not compress non-Latin byte sequences into context that frozen Qwen can use as well as its native tokenizer. This may reflect longer byte sequences, less training coverage in the adapter data, and the fact that Qwen’s own tokenizer already provides strong script-specific units.

This does not imply that byte-level modeling is ineffective. Models trained from scratch or heavily adapted for bytes can be strong (Xue et al., 2022; Clark et al., 2022; Yu et al., 2023). Nor does it rule out larger adapters, longer training, contrastive retrieval-style objectives, partial LM finetuning, or architectures designed to bridge bytes and tokens. It provides evidence against a tempting practical shortcut: attach a small byte adapter to a frozen tokenizer-based LM and expect it to replace native tokenized prompts.

Takeaways for future byte-interface work. Our evidence argues against three simple fixes under this setup. First, making byte vectors close to token embeddings is not enough. Second, predicting token IDs in a separate reconstruction head is not enough, even with oracle token boundaries. Third, injecting byte information deeper into the model helps, but still does not make a small frozen-LM adapter competitive with native tokenization. Future work should treat these as baselines to beat rather than as solved components.

7 Related Work

Byte and character-level language models. ByT5 removes text preprocessing and operates directly on UTF-8 bytes, improving robustness and multilingual coverage in an encoder-decoder setting (Xue et al., 2022). CANINE models Unicode characters with downsampling and upsampling to reduce sequence length (Clark et al., 2022). Charformer learns gradient-based subword tokenization within the model (Tay et al., 2022). MEGABYTE uses multiscale byte modeling with patch-level structure for long byte sequences (Yu et al., 2023). These systems differ from ByteBridge because they train architectures for byte or character inputs, whereas we freeze an existing tokenizer-based LM and train only an external input adapter.

Tokenization and robustness. Prior work has shown that tokenization choices affect robustness, fairness, multilingual efficiency, and downstream performance (Bostrom and Durrett, 2020; Rust et al., 2021; Mielke et al., 2021). ByteBridge focuses on a complementary question: whether an already-tokenized LM can be retrofitted with a tokenizer-free input side while keeping the LM frozen.

Parameter-efficient frozen-LM adaptation. Prompt tuning, prefix tuning, adapters, and LoRA demonstrate that frozen or mostly frozen LMs can be steered by small trainable modules (Li and Liang, 2021; Lester et al., 2021; Houlsby et al., 2019; Hu et al., 2022). Our soft-prefix and KV-prefix experiments borrow this intuition, but the task is harder: the adapter must encode arbitrary raw input bytes into context that substitutes for tokenized prompt information, rather than adapt a task using the model’s native tokenizer.

8 Limitations

This study is scoped to Qwen2.5-0.5B, small adapters, a single-GPU environment, short runs of roughly 1k–2k steps for the main diagnostics, and a synthetic/curated stress benchmark. We do not claim that all byte interfaces for all frozen LMs must fail. We also do not eliminate the target-side tokenizer: supervision and output scoring use Qwen target tokens to make comparisons fair. The study has not yet tested larger models, multiple tokenizer families, substantially larger adapters, long training schedules, contrastive pretraining with negatives, or partial LM finetuning.

The oracle-boundary reconstruction experiments use tokenizer-derived boundaries and therefore are diagnostic upper bounds, not tokenizer-free systems. KV-prefix experiments use a practical cache construction for Qwen’s installed Transformers version; first- N -layer variants use zero K/V tensors for non-injected layers to maintain cache length consistency, which may not be optimal.

9 Conclusion

ByteBridge asks whether a small input-side byte adapter can replace or augment native tokenized input for a frozen tokenizer-based LM. Across a controlled ladder of increasingly expressive interfaces, the answer under our tested setting is no. Byte adapters produce a consistent typo-noise robustness signal and deeper injection improves several stress categories, but clean retention remains too poor and native tokenization continues to dominate non-typo stress buckets. The result suggests that frozen LMs inherit strong dependence on tokenizer-induced token identity, boundary, and positional structure. Future byte-interface retrofitting methods likely need stronger objectives, larger or deeper adapters, partial model adaptation, or architectures trained with byte interfaces from the beginning.

References

Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In Findings of EMNLP, 2020.

- Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. Transactions of the Association for Computational Linguistics, 2022.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Larous-silhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In ICML, 2019.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In ICLR, 2022.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In EMNLP, 2021.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In ACL, 2021.
- Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, and Samson Tan. Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. In ACL, 2021.
- Qwen Team. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115, 2024.
- Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? On the monolingual performance of multilingual language models. In ACL, 2021.
- Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In ICLR, 2022.
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. Transactions of the Association for Computational Linguistics, 2022.
- Lili Yu, Daniel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In NeurIPS, 2023.

A Reproducibility Artifacts

All experiments save configuration files, environment metadata, training metrics, summaries, checkpoints, and per-bucket evaluation outputs. Frozen model trainable parameter counts are zero in all reported LM runs, and frozen checksum delta is zero. The Phase 2 train/validation/test manifests are reused unchanged from Phase 2 through Phase 6.

B KV Cache Probe Details

Qwen2.5-0.5B in the tested Transformers environment uses a DynamicCache. Each layer’s key and value tensors have shape $[B, 2, S, 64]$, reflecting grouped-query attention with 14 attention heads and 2 key/value heads. Manual DynamicCache construction works; legacy tuple-style `past_key_values` does not work in this environment because the model expects cache objects with `get_seq_length`.